

ECOM EXPERTS

Ecom factusol

Manual de usuario, tecnico y de desarrollo

ecom_factusol 1.0.3 · 07/09/2026

Contenido

Paso 1 — Dar de alta el acceso por API en Factusol
Paso 2 — Rellenar la configuracion del modulo
Paso 3 — Probar la conexion
Paso 4 — Activar el envio (desactivar el bloqueo de seguridad)
Paso 5 — Que pasa cuando llega un pedido
Paso 6 — Exportacion a Excel (funcion del escritorio, no del modulo)
Preguntas frecuentes
Documentacion
Skills (en C:\Users\Usuario\gemini\antigravity\global_skills)
Regla central
Pendiente de confirmar en empresa de test
1. Arquitectura general
2. Estructura de ficheros
3. Clases
3.1 FactusolClient (classes/FactusolClient.php)
3.2 FactusolSync (classes/FactusolSync.php)
3.3 FactusolLogger (classes/FactusolLogger.php)
4. Hooks
5. Base de datos
Tabla PREFIX_ecom_factusol_sent (idempotencia)
Tablas de Factusol usadas (via API, no son del modulo)
6. Flujo de envio (7 pasos)
7. Claves de configuracion (Configuration)
KILL_SWITCH
8. Endpoints de la API usados
9. Incertidumbres a validar en empresa de test (antes de produccion)
1. Como esta separado el codigo
2. Como extender
Añadir un campo nuevo a un alta (ej. peso PESART en articulos)
Enviar otra entidad (ej. albaranes F_ALB/F_LAL o facturas F_FAC/F_LFA)
Cambiar el disparador (enviar tras pago en vez de al validar)
3. Como debuggear (activar DEBUG → logs en /logs)
4. Como se calculan los totales
5. Rollback (transaccionalidad manual)
6. Concurrencia
7. Como cambiar la cabecera del token (Bearer vs token)
8. Como probar en empresa de test (paso a paso)
1. Que API usar (importante)
2. Alta y credenciales (una vez)
3. Autenticacion
4. Operaciones genericas /admin

5. Tablas y campos
6. Flujo de envio de un pedido
7. Exportacion a Excel
8. Reglas criticas de seguridad
9. Lo que NO esta confirmado (verificar en empresa de test antes de produccion)

Este modulo envia automaticamente los pedidos de tu tienda PrestaShop a **Factusol**. Cuando entra un pedido, el modulo crea (si no existen ya) el cliente, su direccion de envio y los productos, y despues genera el pedido en Factusol. No tienes que copiar nada a mano.

IMPORTANTE: el modulo se instala **con el envio desactivado** por seguridad. No enviara nada hasta que tu lo actives (ver paso 4). Esto evita que, al instalarlo, se manden pedidos antiguos por error.

Paso 1 — Dar de alta el acceso por API en Factusol

Esto se hace **una sola vez**, dentro del programa Factusol de escritorio:

1. Necesitas primero un **codigoFabricante**: se obtiene dando de alta al desarrollador/integrador como "fabricante" en <https://www.sdelisol.com/api/> (te dan un numero de fabricante y registras un email). Si ya te lo han facilitado, salta este punto.
2. Abre Factusol y ve a: **Archivo** → **Seguridad** → **Acceso por API** → **Nuevo**.
3. Rellena: - **codigoFabricante** (el numero del punto 1) y el **email** asociado. - **Empresa**: la empresa cuyos datos quieres sincronizar. - **Area**: GESTION. - **Permiso**: **Completo** (lectura + escritura). Es imprescindible para que el modulo pueda **crear** clientes, productos y pedidos.
4. Guarda. Factusol te mostrara / asociara los datos que necesitaras en el modulo: **codigoCliente**, **baseDatosCliente** (nombre de la base de datos) y la **password** del acceso API.

Anota estos cuatro datos: codigoFabricante, codigoCliente, baseDatosCliente y password.

Paso 2 — Rellenar la configuracion del modulo

En el back-office de PrestaShop: **Modulos** → **ecom_factusol** → **Configurar**. Rellena:

Campo	Que poner
Codigo de fabricante	El numero de fabricante (paso 1.1)
Codigo de cliente	El codigoCliente que te dio Factusol
Base de datos del cliente	El nombre de la base de datos (baseDatosCliente)
Password	La contraseña del acceso API (se guarda cifrada)
Ejercicio	El año contable, por ejemplo 2026
Serie de pedidos	La serie donde se crearan los pedidos (normalmente 1)
Almacen	El almacen por defecto (normalmente 1)
Familia por defecto	La familia para los productos nuevos que se creen desde la web (por ejemplo WEB)

La **familia por defecto** debe existir ya en Factusol. Crea una familia (por ejemplo "WEB") en el escritorio de Factusol antes de empezar, para que los productos nuevos tengan donde clasificarse.

Guarda los cambios.

Paso 3 — Probar la conexion

Antes de activar nada, pulsa el boton "**Probar conexion**" de la configuracion.

- Si todo esta bien, veras un mensaje de **conexion correcta**: el modulo ha conseguido autenticarse contra Factusol.
- Si falla, revisa que los cuatro datos (fabricante, cliente, base de datos, password) esten bien escritos y que el acceso API tenga permiso **Completo**. Si la conexion de login funciona pero da error al leer datos, avisa al desarrollador: puede que haya que ajustar el tipo de cabecera del token (es un ajuste tecnico conocido, se cambia en un sitio).

Consejo: haz esta primera prueba sobre una **empresa de test** de Factusol, no sobre la real, hasta comprobar que todo funciona.

Paso 4 — Activar el envio (desactivar el bloqueo de seguridad)

El modulo viene con un **bloqueo de seguridad activado** (KILL_SWITCH) para que no envie nada por accidente nada mas instalarlo.

Cuando ya has probado la conexion y estas segur@:

1. En la configuracion, activa la opcion "**Enviar pedidos a Factusol**" (o "Activar envio").
2. Guarda.

A partir de ese momento, cada pedido nuevo se enviara automaticamente. Si quieres parar los envios en cualquier momento, desactiva esa misma opcion.

Paso 5 — Que pasa cuando llega un pedido

Cuando un cliente completa un pedido en la tienda, el modulo hace, en orden:

1. **Ciente:** lo busca en Factusol por NIF, luego por email, luego por nombre. Si no existe, lo crea.
2. **Direccion de envio:** busca la direccion del cliente por calle + codigo postal; si no la tiene, la añade.
3. **Productos:** por cada linea, busca el producto por referencia, luego por codigo de barras (EAN), luego por nombre. Si no existe, lo crea (con la familia por defecto).
4. **Pedido:** crea el pedido en Factusol (cabecera + lineas) con los importes ya calculados.
5. **Marca el pedido como enviado** para no volver a mandarlo nunca (aunque se recargue o cambie de estado).

Cosas a tener en cuenta: - **No se envian pedidos dos veces.** Cada pedido se manda una sola vez. - **El modulo no reenvia pedidos antiguos.** Solo actua sobre pedidos nuevos a partir del momento en que lo activas. - **Los importes se calculan igual que en la tienda** (base, IVA y total por linea). Si detectas un descuadre, avisa al desarrollador con el numero de pedido. - Si algo falla a mitad (por ejemplo, se cae la conexion), el modulo deshace lo que hubiera creado de ese pedido para no dejar documentos incompletos, y lo podras reintentar.

Si necesitas revisar por que un pedido no se envio, pide al desarrollador que active el **modo depuracion**: el modulo guardara un registro detallado en su carpeta `/logs` que ayuda a diagnosticar.

Paso 6 — Exportacion a Excel (funcion del escritorio, no del modulo)

La sincronizacion en tiempo real la hace el modulo por API. Para **cargas masivas puntuales a mano** (por ejemplo, subir un catalogo grande de golpe), Factusol tiene su propio Excel en el programa de escritorio:

- **Exportar a Excel:** en Factusol, `Impresion → Ficheros → Generar en Excel`.
- **Importar masivo desde Excel:** `Utilidades → Importaciones → Ficheros .XLSX`. Los ficheros deben llamarse como la tabla (`ART` para articulos, `CLI` para clientes, `STO` para stock...) y la columna A es el indice.

Esto es util para cargas manuales grandes. Para el dia a dia (pedidos que entran en la tienda) **no hace falta tocar Excel**: el modulo lo envia todo automaticamente.

Preguntas frecuentes

No se ha enviado un pedido, ¿que hago? Comprueba que el envio esta activado (paso 4) y que la conexion funciona (paso 3). Si sigue sin ir, pide activar el modo depuracion y revisa/entrega el fichero de la carpeta `/logs` al desarrollador.

He instalado el modulo y no envia nada. Es lo normal: viene bloqueado por seguridad. Actívalo en el paso 4.

¿Se van a enviar todos mis pedidos antiguos? No. El modulo solo procesa pedidos nuevos desde que lo activas. Los historicos no se tocan (regla de seguridad para no molestar a clientes con pedidos viejos).

Un producto se ha creado en Factusol sin precio de venta. Es correcto: el precio de venta (PVP) y el stock se gestionan en Factusol, no se sincronizan desde la web. El precio que va en el pedido si es el de la venta.

ecom_factusol — Integracion PrestaShop → Factusol

Modulo para enviar pedidos y productos de PrestaShop a Factusol via la **API oficial API DELSOL** (<https://api.sdelisol.com>).

Documentacion

- **PRESTASHOP_A_FACTUSOL.md** — guia completa de integracion (auth, tablas, flujo de envio, mapeo, Excel, incertidumbres).

Skills (en `C:\Users\Usuario\.gemini\antigravity\global_skills`)

- `factusol-api-auth` — autenticacion, modelo CRUD /admin, clase PHP `FactusolClient` .
- `factusol-articulos` — buscar/crear productos (F_ART) por referencia/EAN/nombre.
- `factusol-clientes` — buscar/crear clientes (F_CLI) por NIF/email/nombre.
- `factusol-direcciones` — direcciones de envio (F_DIR).
- `factusol-pedidos` — crear pedido cabecera F_PCL + lineas F_LPC.
- `factusol-prestashop-sync` — orquestacion completa del envio + Excel.

Regla central

Todo producto/cliente se busca por **referencia** → **EAN** → **nombre**; si no existe, se crea antes de enviar el pedido.

Pendiente de confirmar en empresa de test

Cabecera del token (Bearer vs `token`), nombre real de `F_DIR` , campos minimos de alta, numeracion de documentos. Ver seccion 9 de la guia.

ecom_factusol — Documentacion Tecnica

Modulo PrestaShop 8/9 que envia pedidos (y da de alta los clientes, direcciones y productos que necesite) a **Factusol** a traves de la **API oficial API DELSOL** (<https://api.sdelisol.com>, documentacion en <https://apidoc.sdelisol.com>).

- **Autor:** Ecom Experts (ecomyseo@gmail.com)
- **Licencia:** AFL-3.0 — Copyright 2026
- **Compatibilidad:** PrestaShop 1.7 / 8 / 9 · PHP 7.4 y 8.x
- **Dependencias externas:** ninguna (cURL nativo, sin Composer, sin Guzzle/Symfony)

1. Arquitectura general

El modulo se apoya en el **modelo CRUD generico** de la API DELSOL. La API **no** tiene endpoints por entidad (`/articulos`, `/clientes`, `/pedidos`): todo se hace enviando `tabla + filtro SQL + registro` contra rutas `/admin/*` sobre las tablas nativas de Factusol `F_*`.

Regla de negocio central:

Todo cliente, direccion y producto se busca por **referencia** → **EAN** → **nombre** (segun la entidad); **si no existe, se crea** antes de montar el pedido.

Flujo de alto nivel cuando entra un pedido valido en PrestaShop:

```
Hook de pedido PS
-> Ecom_Factusol::hook... (comprueba KILL_SWITCH e idempotencia)
-> FactusolSync::enviarPedido()
-> FactusolClient (autenticar + CRUD /admin)
-> Factusol (F_CLI, F_DIR, F_ART, F_PCL, F_LPC)
-> marca el pedido como enviado en ecom_factusol_sent
-> FactusolLogger (si DEBUG activo) escribe en /logs
```

2. Estructura de ficheros

```
ecom_factusol/
├─ ecom_factusol.php      Clase principal Ecom_Factusol (install, hooks, config)
├─ index.php             Redireccion de seguridad
├─ config.xml
├─ logo.png
├─ classes/
│  ├─ FactusolClient.php  Transporte HTTP + CRUD generico /admin (auth, cargaTabla, escribirRegistro...)
│  ├─ FactusolSync.php    Orquestacion: resolver/crear cliente, direccion, articulos y crear el pedido
│  ├─ FactusolLogger.php  Logger a fichero (solo si DEBUG activado)
│  └─ index.php
├─ logs/                 Ficheros .log (se crean solo con DEBUG activo)
│  └─ index.php
├─ sql/
│  ├─ install.php         Crea ecom_factusol_sent
│  └─ index.php
├─ views/
│  ├─ templates/admin/    Plantillas de configuracion (boton probar conexion)
│  └─ ...
├─ translations/         .xlf (sistema nuevo de traduccion)
└─ docs/
   ├─ DOC_TECNICO.md      (este fichero)
   ├─ DOC_DESARROLLO.md
   └─ GUIA_USUARIO.md
```

Cada directorio recursivo lleva su `index.php` con cabecera y `header('Location: ../'); exit;` segun norma PrestaShop del proyecto.

3. Clases

3.1 FactusolClient (classes/FactusolClient.php)

Cliente HTTP puro contra la API DELSOL. Sin logica de negocio. Nativo (cURL), sin namespace, protegido con `if (!defined('_PS_VERSION_')) { exit; }.`

Constantes de transporte (los dos puntos a validar en test):

Constante	Valor por defecto	Que controla
<code>BASE_URL</code>	<code>https://api.sdelisol.com</code>	Host base
<code>AUTH_HEADER_BEARER</code>	<code>true</code>	<code>true</code> = Authorization: Bearer <token> · <code>false</code> = cabecera literal token: <token>
<code>LOGIN_PATH</code>	<code>/login/Autenticar</code>	Probar tambien <code>/login/autenticar</code>

Constructor: `new FactusolClient($codigoFabricante, $codigoCliente, $baseDatosCliente, $passwordPlain, $ejercicio).`

Metodos publicos:

Metodo	Ruta API	Descripcion
<code>authenticate()</code>	<code>POST /login/Autenticar</code>	Autentica (password en Base64), cachea el token JWT. Lanza <code>Exception</code> si falla
<code>cargaTabla(\$tabla,\$filtro)</code>	<code>POST /admin/CargaTabla</code>	Devuelve N filas ya normalizadas (array asociativo <code>COL=>dato</code>)
<code>leerRegistro(\$tabla,\$filtro)</code>	<code>GET /admin/LeerRegistro/...</code>	Una fila o <code>null</code>
<code>lanzarConsulta(\$sql)</code>	<code>POST /admin/LanzarConsulta</code>	SELECT libre (JOIN, MAX...)
<code>escribirRegistro(\$tabla,\$campos)</code>	<code>POST /admin/EscribirRegistro</code>	INSERT. Devuelve <code>['ok'=>bool, 'raw'=>...]</code>
<code>actualizarRegistro(\$tabla,\$campos)</code>	<code>POST /admin/ActualizarRegistro</code>	UPDATE (incluir columna clave)
<code>maxCodigo(\$col,\$tabla,\$where)</code>	via <code>LanzarConsulta</code>	<code>MAX(col)</code> como entero (numeracion)

Internos relevantes: - `withRetry()` — reintenta **una vez** si la respuesta es `Unauthorized` (token expirado): descarta el token, re-autentica y repite la llamada. - `normalizeRows()` — convierte el formato de respuesta `[[{columna,dato}]]` en arrays asociativos `COL => valor`. - `authHeaders()` — construye las cabeceras segun `AUTH_HEADER_BEARER`. - `curl()` — transporte, `CURLOPT_TIMEOUT = 30`, `Content-Type: application/json`.

3.2 FactusolSync (classes/FactusolSync.php)

Orquesta el envio. Contiene los helpers de resolucion/creacion y el metodo de alto nivel. Recibe un `FactusolClient` ya construido.

- `factusolResolverCliente($fs, $cli)` → busca `F_CLI` por **NIF** → **email** → **nombre**; si no existe crea con `CODCLI` correlativo (`MAX+1`, 6 digitos). Devuelve `CODCLI`.
- `factusolResolverDireccion($fs, $codcli, $dir)` → busca en `F_DIR` del cliente por **domicilio + CP**; si no coincide crea con `CODDIR = MAX(CODDIR del cliente)+1`. Devuelve `CODDIR`.
- `factusolResolverArticulo($fs, $prod, $familiaDefecto)` → busca `F_ART` por **CODART (=referencia)** → **EANART** → **DESART**; si no existe crea usando la referencia como `CODART` (o correlativo `Wxxxxxx`) y `FAMART = familia por defecto`. Devuelve `CODART`.
- `factusolCrearPedido($fs, $cab, $lineas)` → calcula numero (`MAX(CODPCL)+1` de la serie), calcula totales/IVA en PHP, inserta cabecera `F_PCL` y lineas `F_LPC` con **rollback manual** si falla alguna linea. Devuelve `['serie', 'numero', 'total']`.
- `factusolEnviarPedido($fs, $order, $familiaDefecto)` → orquestador de los 7 pasos.

3.3 FactusolLogger (classes/FactusolLogger.php)

Logger a fichero, activo **solo** si la configuracion `ECOM_FACTUSOL_DEBUG` esta a 1. Escribe en `_PS_MODULE_DIR_ . 'ecom_factusol/logs/'` (nunca `/tmp` ni `/var/logs`). Un fichero por dia (p.ej. `factusol-2026-05-07.log`). Registra: inicio de envio, cada llamada relevante (tabla, filtro, respuesta OK/KO), `CODCLI/CODDIR/CODART` resueltos, numero de pedido generado, y errores con traza. Nunca vuelca el `password` en claro.

4. Hooks

El modulo se engancha al ciclo de vida del pedido. El disparo del envio se hace en un unico punto para respetar la idempotencia:

Hook	Uso
<code>actionValidateOrder</code>	Punto principal: pedido recién creado y validado → dispara el envio (tras chequear KILL_SWITCH e idempotencia)
<code>actionPaymentConfirmation</code> (alternativa)	Si se prefiere enviar solo tras pago confirmado
<code>actionAdminControllerSetMedia</code> / render de config	Back-office: pinta la pantalla de configuracion y el boton "Probar conexion"

Todo hook declarado tiene su metodo `public function hookNombre(array $params) {}`. Antes de insertar cualquier hook se comprueba si ya existe (norma del proyecto).

Idempotencia en el hook: antes de llamar a `FactusolSync` se comprueba `ecom_factusol_sent`. Si el pedido ya esta, se sale sin hacer nada. Tras un envio correcto se inserta la fila con `serie+numero`. **Nunca** se recorre el historico de pedidos para reenviar (regla anti-scan del proyecto: ventana maxima 24h / seed).

5. Base de datos

Tabla `PREFIX_ecom_factusol_sent` (idempotencia)

Registro de pedidos ya enviados para no reenviar. Se crea en `install()` (y se auto-repara si falta).

Columna	Tipo	Descripcion
<code>id_ecom_factusol_sent</code>	INT PK AUTO	Clave
<code>id_order</code>	INT, UNIQUE	Pedido PrestaShop (clave anti-duplicado)
<code>fs_serie</code>	VARCHAR(8)	Serie Factusol (<code>TIPPCL</code>)
<code>fs_numero</code>	INT	Numero de pedido Factusol (<code>CODPCL</code>)
<code>fs_total</code>	DECIMAL(12,2)	Total enviado
<code>status</code>	VARCHAR(16)	<code>OK</code> / <code>ERROR</code>
<code>date_add</code>	DATETIME	Fecha del envio

El `UNIQUE(id_order)` es la barrera dura anti-duplicado. Se lee con `SELECT` + `INSERT` con try/catch (no se confia en `INSERT IGNORE` bajo concurrencia).

Tablas de Factusol usadas (via API, no son del modulo)

Tabla	Entidad	Campos clave
<code>F_ART</code>	Articulos	<code>CODART</code> (PK, = referencia), <code>EANART</code> (EAN13), <code>DESART</code> (nombre), <code>FAMART</code> (familia, obligatoria), <code>TIVART</code> (tipo IVA), <code>PCOART</code> (coste, NO PVP)
<code>F_CLI</code>	Clientes	<code>CODCLI</code> (PK), <code>NIFCLI</code> , <code>NOFCLI</code> , <code>EMACLI</code> , <code>TELCI</code> , <code>DOMCLI/POBCLI/PROCLI/CPOCLI/PAICLI</code>
<code>F_DIR</code>	Direcciones	<code>CLIDIR</code> (=CODCLI)+ <code>CODDIR</code> (correlativo por cliente), <code>NOMDIR/DOMDIR/CPODIR/POBDIR/PRODIR/TELDIR/PCODIR/PAIDIR</code>
<code>F_PCL</code>	Pedido cabecera	<code>TIPPCL</code> (serie)+ <code>CODPCL</code> (numero), <code>FECPL</code> , <code>CLIPCL</code> (=CODCLI), <code>OBPPCL</code> (=CODDIR), <code>ALMPCL</code> , <code>FOPPCL</code> , <code>TOTPCL</code>
<code>F_LPC</code>	Pedido lineas	<code>TIPLPC</code> + <code>CODLPC</code> (enlace), <code>POSLPC</code> , <code>ARTLPC</code> (=CODART), <code>DESLPC</code> , <code>CANLPC</code> , <code>PRELPC</code> (sin IVA), <code>IVALPC</code> / <code>TIVLPC</code> , <code>DT1LPC</code> , <code>TOTLPC</code>

El PVP no esta en `F_ART` (vive en `F_LTA`); el stock tampoco (vive en `F_STO`). Para vender no se necesitan: el precio va en la linea del pedido (`PRELPC`).

6. Flujo de envio (7 pasos)

```

0. Autenticar                -> token JWT (cachear; renovar si Unauthorized)
1. Cliente                   -> buscar F_CLI por NIF -> email -> nombre; si no, crear (CODCLI = MAX+1) -> CODCLI
2. Direccion                 -> buscar F_DIR del cliente por domicilio+CP; si no, crear (CODDIR = MAX por cliente +1) -> CODDIR
3. Articulos (por linea)     -> buscar F_ART por CODART -> EANART -> DESART; si no, crear -> CODART
4. Numero de pedido          -> MAX(CODPCL)+1 de la serie
5. Cabecera                  -> crear F_PCL (CLIPCL, OBRPCL, ALMPCL, FOPPCL, TOTPCL calculado en PHP)
6. Lineas                    -> crear F_LPC (una por linea; rollback de cabecera+lineas si falla alguna)
7. Idempotencia              -> guardar serie+numero en ecom_factusol_sent (PrestaShop)

```

Totales: la API no recalcula IVA ni totales. Se calculan en PHP: `base = cantidad * precio_sin_iva * (1 - dto/100)`, `total_linea = round(base * (1 + iva/100), 2)`, `total_pedido = suma de lineas`.

Rollback: si una linea falla despues de crear la cabecera, se borran las lineas y la cabecera con `BorrarRegistros` para no dejar pedidos huérfanos.

7. Claves de configuracion (Configuration)

Prefijo `ECOM_FACTUSOL_`. Las credenciales se guardan cifradas.

Clave	Descripcion
<code>ECOM_FACTUSOL_COD_FAB</code>	codigoFabricante (int)
<code>ECOM_FACTUSOL_COD_CLI</code>	codigoCliente (int)
<code>ECOM_FACTUSOL_BD</code>	baseDatosCliente (nombre de la BD del cliente en Factusol)
<code>ECOM_FACTUSOL_PASS</code>	password del acceso API (guardada cifrada; se envia en Base64 en el login)
<code>ECOM_FACTUSOL_EJERCICIO</code>	Ejercicio contable / año (string, p.ej. "2026")
<code>ECOM_FACTUSOL_SERIE</code>	Serie de pedidos (<code>TIPPCL</code>), por defecto <code>1</code>
<code>ECOM_FACTUSOL_ALMACEN</code>	Almacen (<code>ALMPCL</code>), por defecto <code>1</code>
<code>ECOM_FACTUSOL_FAMILIA</code>	Familia por defecto para altas de articulos web (<code>FAMART</code>), p.ej. <code>WEB</code>
<code>ECOM_FACTUSOL_ENABLED</code>	Interruptor de envio. Con el <code>KILL_SWITCH</code> activo (por defecto) NO se envia hasta que el usuario lo active
<code>ECOM_FACTUSOL_DEBUG</code>	Activa el logging a <code>/logs</code>
<code>ECOM_FACTUSOL_AUTH_HEADER</code>	(opcional) Selecciona Bearer vs <code>token</code> sin tocar codigo

KILL_SWITCH

Por norma anti-catastrofe del proyecto, el modulo se entrega con el envio **desactivado por defecto**. Existe una constante `KILL_SWITCH = true` en la clase principal que cortocircuita todos los puntos de envio al inicio (`if (self::KILL_SWITCH) { return; }`), y/o la clave `ECOM_FACTUSOL_ENABLED = 0`. El usuario activa el envio conscientemente desde la configuracion (o cambiando la constante a `false`) una vez validada la conexion en empresa de test.

8. Endpoints de la API usados

Operacion	Metodo + ruta	Cuerpo
Login	POST /login/Autenticar	{codigoFabricante,codigoCliente,baseDatosCliente,password(Base64)}
Buscar N	POST /admin/CargaTabla	{ejercicio,tabla,filtro}
Leer 1	GET /admin/LeerRegistro/{ejercicio}/{tabla}/{filtro}	—
SELECT libre	POST /admin/LanzarConsulta	{ejercicio,consulta}
Crear	POST /admin/EscribirRegistro	{ejercicio,tabla,registro:[{columna,dato}]}
Actualizar	POST /admin/ActualizarRegistro	igual + columna clave
Borrar (rollback)	GET /admin/BorrarRegistros/{ejercicio}/{tabla}/{filtro}	— (PELIGROSO)

Respuesta general: {"resultado":[...],"respuesta":"OK"}. resultado es array de filas; cada fila array de {columna,dato}. Token caducado → respuesta:"Unauthorized".

9. Incertidumbres a validar en empresa de test (antes de produccion)

Estos puntos estan inferidos por patron/analogia y **deben** confirmarse contra un servidor real antes de activar el envio en produccion:

1. **Cabecera del token:** Authorization: Bearer <token> vs cabecera literal token: <token> → conmutar con AUTH_HEADER_BEARER.
2. **Ruta de login:** login/Autenticar vs login/autenticar (mayusculas).
3. **baseDatosCliente**: formato/prefijo exacto esperado por Factusol.
4. **Nombre real de la tabla de direcciones** F_DIR y el campo PAIDIR (inferidos).
5. **Campos minimos obligatorios** para dar de alta articulo / cliente / direccion.
6. **Numeracion de documentos:** si EscribirRegistro respeta el contador de serie o hay que calcular MAX(CODPCL)+1 (se asume manual).
7. Si OBRPCL guarda el CODDIR numerico o el NOMDIR.
8. Tabla de lineas de factura si se amplia a facturas: F_LFA vs F_LFC.

Prueba minima recomendada: (a) autenticar y fijar la cabecera correcta; (b) CargaTabla F_ART (ver formato de respuesta); (c) CargaTabla F_DIR (confirmar nombre de tabla); (d) crear articulo y cliente de prueba (descubrir campos obligatorios); (e) crear un pedido F_PCL + F_LPC y verificarlo en el escritorio de Factusol.

ecom_factusol — Guia del Desarrollador

Guia interna para mantener y extender el modulo. Requiere haber leído el `DOC_TECNICO.md`. Todo el codigo de negocio vive en `classes/FactusolClient.php`, `classes/FactusolSync.php` y `classes/FactusolLogger.php`.

1. Como esta separado el codigo

- **FactusolClient** = transporte + CRUD generico. No sabe nada de PrestaShop. Si cambia la API DELSOL (rutas, cabeceras, formato de respuesta), se toca **solo aqui**.
- **FactusolSync** = reglas de negocio (resolver/crear, calculo de totales, rollback). Depende de **FactusolClient** pero no de la clase del modulo.
- **Ecom_Factusol** (fichero principal) = pegamento PrestaShop: config, hooks, idempotencia (`ecom_factusol_sent`), KILL_SWITCH y logging.

Regla: la logica de red va en **FactusolClient**; la logica de datos en **FactusolSync**; lo que toque **Configuration**, hooks o BD de PrestaShop va en el fichero principal.

2. Como extender

Añadir un campo nuevo a un alta (ej. peso `PESART` en articulos)

1. En `factusolResolverArticulo()` añade la columna al array `$campos : $campos['PESART'] = (float) $prod['weight'];`
2. Asegurate de que quien construye `$order['lineas']` (el hook) rellene ese dato desde el producto PrestaShop.
3. No hace falta tocar `FactusolClient : escribirRegistro` acepta cualquier columna.

Enviar otra entidad (ej. albaranes `F_ALB / F_LAL` o facturas `F_FAC / F_LFA`)

Las tablas hermanas siguen el mismo patron `cabecera + lineas` con prefijos de campo distintos. Duplica `factusolCrearPedido()` cambiando: - Tabla cabecera (`F_ALB / F_FAC`) y sus campos (`TIP* / COD* / CLI* / TOT*`). - Tabla lineas (`F_LAL / F_LFA` o `F_LFC` — **confirmar cual** en test) y sus campos. Mantén el mismo esquema de numeracion `MAX+1`, calculo de totales y rollback.

Cambiar el disparador (enviar tras pago en vez de al validar)

Mueve la llamada de `actionValidateOrder` a `actionPaymentConfirmation`. La idempotencia por `id_order` sigue protegiendo de doble envio si ambos hooks estuvieran activos.

3. Como debuggear (activar DEBUG → logs en /logs)

1. Back-office → configuracion del modulo → activa **"Modo depuracion"** (`ECOM_FACTUSOL_DEBUG = 1`).
2. **FactusolLogger** empieza a escribir en `ecom_factusol/logs/factusol-YYYY-MM-DD.log`. La carpeta `/logs` se crea automaticamente si no existe.
3. Reproduce el caso (haz un pedido de prueba o pulsa "Probar conexion").
4. Revisa el `.log`: veras el paso a paso (autenticacion, cada `CargaTabla / Escribir`, `CODCLI/CODDIR/CODART` resueltos, numero de pedido, y el cuerpo `raw` de las respuestas KO).

Puntos donde conviene loggear al depurar: - Respuesta cruda del login (para ver si el token llega o da `Unauthorized`). - El `filtro` SQL exacto de cada busqueda (fallos por comillas sin escapar). - El `raw` de cada `escribirRegistro` que devuelva `ok=false` (campos obligatorios).

El logger **nunca** escribe el `password` en claro. Si añades logs nuevos, respeta esa regla y no vuelques credenciales.

4. Como se calculan los totales

La API **no** recalcula IVA ni totales; si tus numeros no cuadran, el documento sale descuadrado en Factusol. El calculo esta en `factusolCrearPedido()`:

```
base_linea = cantidad * precio_sin_iva * (1 - dto1/100)
total_linea = round(base_linea * (1 + iva/100), 2) -> se guarda en TOTLPC
total_pedido = round(suma(total_linea), 2) -> se guarda en TOTPCL
```

- `precio_sin_iva` = `PRELPC` (precio unitario SIN IVA), viene de PrestaShop como `unit_price_tax_excl`.
- `iva` = porcentaje (21, 10, 4...), viene de `tax_rate`.
- `dto1` = descuento de linea en %; si no hay, 0.
- El redondeo por linea antes de sumar evita descuadres de centimos. Si Factusol redondea distinto (a nivel de base imponible agrupada por tipo de IVA), ajusta aqui — es el unico sitio del calculo.

5. Rollback (transaccionalidad manual)

No hay transaccion atomica cabecera+lineas. `factusolCrearPedido()` : 1. Inserta cabecera `F_PCL`. Si falla → `Exception`, no se insertó nada. 2. Inserta lineas `F_LPC` una a una. Si **cualquier** linea falla: - Borra todas las lineas del pedido: `BorrarRegistros .../F_LPC/ TIPLPC='serie' AND CODLPC=numero`. - Borra la cabecera: `BorrarRegistros .../F_PCL/ TIPPCL='serie' AND CODPCL=numero`. - Re-lanza la excepcion.

Asi no quedan pedidos huérfanos. **Cuidado con `BorrarRegistros`**: borra TODO lo que matchee el filtro; el filtro siempre debe llevar serie + numero exactos.

Como el rollback ocurrio, el pedido **no** se marca como enviado en `ecom_factusol_sent` con status OK; se registra `ERROR` para diagnostico y se puede reintentar mas tarde.

6. Concurrencia

Dos riesgos por el uso de `MAX+1`:

1. **Numeracion de pedido (`CODPCL`)**: dos pedidos simultaneos pueden calcular el mismo `MAX(CODPCL)+1`. Mitigacion: serializar el envio (procesar en cola/cron secuencial) o **reintentar** el alta con `numero+1` si el insert choca.
2. **`CODCLI` / `CODDIR` correlativos**: mismo problema al crear clientes/direcciones a la vez. Reintenta el alta con el siguiente codigo si el insert falla por clave duplicada.

En PrestaShop, ademas, protege el disparo con la idempotencia por `id_order` (`SELECT` + `INSERT` con try/catch, **no** `INSERT IGNORE` bajo concurrencia) y con un guard estatico en memoria por peticion para evitar doble hook en la misma request.

7. Como cambiar la cabecera del token (Bearer vs token)

Es la incertidumbre n°1. En `FactusolClient`:

```
const AUTH_HEADER_BEARER = true; // true -> Authorization: Bearer <token>
                                // false -> cabecera literal "token: <token>"
```

`authHeaders()` construye la cabecera segun esta constante:

```
$h[] = self::AUTH_HEADER_BEARER
      ? ('Authorization: Bearer ' . $this->token)
      : ('token: ' . $this->token);
```

Para probar sin editar codigo, se puede exponer la clave `ECOM_FACTUSOL_AUTH_HEADER` en la config y pasarla al cliente. Procedimiento de prueba: 1. Deja `AUTH_HEADER_BEARER = true`, pulsa "Probar conexion". 2. Si login OK pero las llamadas `/admin/*` dan `Unauthorized`, cambia a `false` y vuelve a probar. 3. Fija el valor que funcione y no lo toques mas.

Igual criterio para `LOGIN_PATH` (`/login/Autenticar` vs `/login/autenticar`).

8. Como probar en empresa de test (paso a paso)

Nunca pruebes por primera vez sobre la BD real. Usa una **empresa de test** en Factusol (o el ejercicio de pruebas).

1. **Credenciales de test:** da de alta el acceso API sobre la empresa de test (Archivo > Seguridad > Acceso por API) con area GESTION y permiso Completo.
2. **Config del modulo** con esas credenciales y `ejercicio` de test. Deja el **KILL_SWITCH activo** todavia.
3. **Activa DEBUG** para tener logs.
4. **Probar conexion** (boton de la config) → debe autenticar y devolver OK. Si no, ajusta `AUTH_HEADER_BEARER` / `LOGIN_PATH` (ver §7).
5. **Lectura:** verifica en logs un `CargaTabla F_ART` y un `CargaTabla F_DIR` (confirma el nombre real de la tabla de direcciones y el formato de respuesta).
6. **Alta controlada:** crea a mano (o con un pedido de test) un articulo y un cliente; revisa en el escritorio de Factusol que aparecen y con que campos minimos (descubre los obligatorios reales).
7. **Pedido completo:** lanza un pedido de test → comprueba en el escritorio de Factusol la cabecera `F_PCL` y las lineas `F_LPC` , y que el `TOTPCL` cuadra.
8. **Idempotencia:** repite el disparo del mismo pedido → NO debe crear un segundo documento (la tabla `ecom_factusol_sent` lo bloquea).
9. **Solo cuando 4-8 sean correctos:** desactiva el KILL_SWITCH / activa `ECOM_FACTUSOL_ENABLED = 1` en la empresa real.

Prohibido recorrer el historico de pedidos para "reenviar los antiguos": ventana maxima de reproceso 24h y/o seed previo (regla anti-scan del proyecto). Si hay que cargar historico, hazlo con la exportacion/importacion Excel del escritorio, no con un scan que dispare la API.

Enviar pedidos y productos desde PrestaShop a Factusol (API DELSOL)

Documento de integracion. Fuente: API oficial **API DELSOL** (<https://api.sdelisol.com> , doc <https://apidoc.sdelisol.com>). Skills asociados en `global_skills`: `factusol-api-auth` , `factusol-articulos` , `factusol-clientes` , `factusol-direcciones` , `factusol-pedidos` , `factusol-prestashop-sync` .

Regla de negocio central: **todo producto y cliente se busca por referencia, EAN o nombre; si no existe, se crea antes de enviar el pedido.**

1. Que API usar (importante)

Hay dos "APIs Factusol". Usa SOLO la oficial:

	API DELSOL (oficial)	zoilomora/factusol-rest-api
Host	<code>https://api.sdelisol.com</code>	app de terceros auto-alojada
Escritura	SI (CRUD completo)	NO (solo lectura)
Uso aqui	La que se usa	Solo diccionario de campos F_*

La API oficial **no tiene endpoints por entidad** (`/articulos` , `/clientes` ...). Es un **CRUD generico** sobre las tablas nativas `F_*` variando tabla + filtro SQL + registro.

2. Alta y credenciales (una vez)

1. **Desarrollador**: alta como "fabricante" en `https://www.sdelisol.com/api/` → `codigoFabricante` + email.
2. **Cliente (en Factusol)**: Archivo > Seguridad > Acceso por API > Nuevo → `codigoFabricante` + email, **empresa, area = GESTION, permiso Completo (lectura+escritura)**.

Configuracion del modulo (guardar cifrada en `Configuration`): `codigoFabricante` , `codigoCliente` , `baseDatosCliente` , `password` , `ejercicio` (año, string), serie de pedidos, almacen, familia por defecto para altas web.

3. Autenticacion

```
POST https://api.sdelisol.com/login/Autenticar
{ "codigoFabricante":123, "codigoCliente":456, "baseDatosCliente":"NOMBREBD", "password":"<BASE64>" }
```

OK → `{"resultado":"<tokenJWT>","respuesta":"OK"}` · Caducado → `{"resultado":"","respuesta":"Unauthorized"}` (re-autenticar).

El token va en cada llamada. **A confirmar contra el servidor real**: cabecera `Authorization: Bearer <token>` o cabecera literal `token: <token>`. La clase `FactusolClient` (skill `factusol-api-auth`) trae un conmutador `AUTH_HEADER_BEARER` para ambos casos.

4. Operaciones genericas /admin

Operacion	Metodo + ruta
Buscar N	POST /admin/CargaTabla {ejercicio,tabla,filtro}
Leer 1	GET /admin/LeerRegistro/{ejercicio}/{tabla}/{filtro}
SELECT libre	POST /admin/LanzarConsulta {ejercicio,consulta}
Crear	POST /admin/EscribirRegistro {ejercicio,tabla,registro:[{columna,dato}]}
Actualizar	POST /admin/ActualizarRegistro (incluir columna clave)
Borrar	GET /admin/BorrarRegistros/{ejercicio}/{tabla}/{filtro} (PELIGROSO)

Respuesta: `resultado` = array de filas; cada fila = array de `{columna,dato}` .

5. Tablas y campos

Articulos `F_ART` — `CODART` (PK, = referencia PS), `EANART` (EAN13 = ean13 PS), `DESART` (nombre), `FAMART` (familia, obligatoria), `TIVART` (tipo IVA), `PCOART` (coste, NO es PVP). PVP → `F_LTA` ; stock → `F_STO` .

Clientes `F_CLI` — `CODCLI` (PK), `NIFCLI`, `NOFCLI`, `EMACLI`, `TELCLI`, `DOMCLI/POBCLI/PROCLI/CPOCLI/PAICLI` .

Direcciones `F_DIR` — clave `CLIDIR` (=CODCLI)+ `CODDIR` (correlativo por cliente); `NOMDIR/DOMDIR/CPDIR/POBDIR/PRODIR/TELDIR/PCODIR` .

Pedido cabecera `F_PCL` — `TIPPCL` (serie)+ `CODPCL` (numero), `FECPL`, `CLIPCL` (=CODCLI), `OBRPCL` (=CODDIR), `ALMPCL`, `FOPPCL`, `TOTPCCL` .

Pedido lineas `F_LPC` — `TIPLPC` + `CODLPC` (enlace), `POSLPC`, `ARTLPC` (=CODART), `DESLPC`, `CANLPC`, `PRELPC` (sin IVA), `IVALPC` / `TIVLPC`, `DT1LPC`, `TOTLPC` .

6. Flujo de envio de un pedido

```
0. Autenticar
1. Cliente:  buscar F_CLI por NIF -> email -> nombre; si no, crear (CODCLI = MAX+1)
2. Direccion: buscar F_DIR del cliente por domicilio+CP; si no, crear (CODDIR = MAX por cliente +1)
3. Articulos: por cada linea buscar F_ART por CODART -> EANART -> DESART; si no, crear
4. Numero:   MAX(CODPCL)+1 de la serie
5. Cabecera: crear F_PCL (totales calculados en PHP)
6. Lineas:   crear F_LPC (una por linea; rollback si falla)
7. Guardar serie+numero en PS (idempotencia)
```

El codigo PHP completo (clase `FactusolClient` + helpers `factusolResolverCliente/Direccion/Articulo`, `factusolCrearPedido`, `factusolEnviarPedido`) esta en los skills. Copiar `FactusolClient` a `classes/` del modulo y los helpers a un `classes/FactusolSync.php` .

Ejemplo de invocacion desde un hook de PrestaShop:

```
$fs = new FactusolClient($cfgFab, $cfgCli, $cfgBd, $cfgPass, $cfgEjercicio);
$order = array(
    'cliente' => array('nif'=>$nif,'email'=>$email,'company_or_name'=>$nombre,
        'address1'=>$dir,'city'=>$ciudad,'state'=>$prov,'postcode'=>$cp,
        'phone'=>$tel,'country_iso'=>$iso),
    'direccion' => array('name'=>'Envio','address1'=>$dirEnv,'postcode'=>$cpEnv,
        'city'=>$ciudadEnv,'state'=>$provEnv,'phone'=>$tel,'contact'=>$nombre),
    'lineas' => $lineas, // ver mapeo abajo
    'serie'=>'1','almacen'=>'1','forma_pago'=>'1',
);
$res = factusolEnviarPedido($fs, $order, 'WEB'); // ['serie','numero','total']
```

Construir `$lineas` desde el pedido PS (una por producto):

```
$lineas[] = array(
    'reference'=>$p['reference'], 'ean13'=>$p['ean13'], 'name'=>$p['name'],
    'wholesale_price'=>$p['wholesale_price'], 'tiva'=>0,
    'cantidad'=>$p['product_quantity'],
    'precio_sin_iva'=>$p['unit_price_tax_excl'],
    'iva'=>$p['tax_rate'], 'dto1'=>0,
);
```

7. Exportacion a Excel

- **La API REST no exporta Excel** (devuelve JSON).
- Factusol tiene Excel **en el escritorio**: importar masivo (Utilidades > Importaciones > Ficheros .XLSX; ficheros con nombre = tabla: **ART**, **CLI**, **STO**, columna A = indice). Util para cargas puntuales a mano, no para la sync en tiempo real.

Implementado en el modulo (`classes/FactusolExport.php`): dos botones en la configuracion generan **ART.xlsx** (articulos) y **CLI.xlsx** (clientes) en el formato de importacion de Factusol, usando **PhpSpreadsheet que ya incluye el core de PrestaShop** (`vendor/phpoffice/phpspreadsheet` , sin anadir Composer al modulo). Los descargas y los importas a mano en Factusol escritorio. - Layout **ART** (documentado): A=CODART (referencia), B=EANART (ean13), E=FAMART (familia por defecto), F=DESART (nombre), J=TIVART (0), K=PCOART (precio coste). Codigos/EAN se escriben como **texto** para preservar ceros a la izquierda. - Layout **CLI** (orden razonable, **VERIFICAR** contra el asistente de importacion de Factusol): A=CODCLI, B=NIF, C=nombre, D=domicilio, E=poblacion, F=provincia, G=CP, H=telefono, I=email. - La columna A es el indice obligatorio; no se generan filas vacias. Opcion **cab** para incluir fila de cabecera (por defecto sin cabecera, datos desde la fila 1). - **Exportar pedidos**: pestaña "Factusol Excel" (menu Pedidos) con filtros nº exacto, rango de nº y rango de fechas. Es export de datos (Factusol no importa documentos por Excel). Filtra por tienda en multishop. - **Memoria**: PhpSpreadsheet mantiene toda la hoja en memoria. En catalogos/historicos muy grandes (decenas de miles de filas) puede requerir subir `memory_limit`; los metodos aceptan un `$limit` opcional para acotar. - **Cron de exportacion**: front controller `controllers/front/cron.php` protegido por token secreto (`ECOM_FACTUSOL_CRONTOKEN`). URL: `index.php?fc=module&module=ecom_factusol&controller=cron&token=SECRETO&days=N&what=all|art|cli|ped` . Genera los XLSX de los ultimos N dias (max 365) en `exports/` en segundo plano (sube memoria/tiempo). Se ve en la config con boton "Generar ahora", dias por defecto y lista de ficheros. NO envia nada a la API (solo exporta a fichero local: cumple la regla anti-scan). - **Privacidad de exports/** : los XLSX contienen datos personales. La carpeta lleva `index.php` + `.htaccess` (`Deny from all`) y los nombres llevan sufijo aleatorio. **En Nginx el .htaccess NO se aplica**: anade una regla para denegar el acceso directo, p.ej. `location ~* /modules/ecom_factusol/exports/ { deny all; }` . Las descargas desde el back-office van siempre por la pagina de config (autenticada).

8. Reglas criticas de seguridad

- **Idempotencia**: guarda `serie+numero` en PS; no reenvies. NUNCA recorras el historico de pedidos disparando envios a Factusol. Ventana maxima de re-procesado **24h** y/o **seed** previo (regla anti-scan del CLAUDE.md).
- **Concurrencia**: `MAX+1` de codigos puede colisionar con envios simultaneos → serializar o reintentar.
- **Totales**: la API no recalcula IVA/totales; calculalos en PHP o el documento saldra descuadrado.
- **Sin transaccion**: si fallan las lineas tras crear la cabecera, borra ambas (rollback manual, ya incluido en `factusolCrearPedido`).
- **SQL**: escapa comillas en los filtros (`str_replace("'", "''", ...)` / `pSQL`).

9. Lo que NO esta confirmado (verificar en empresa de test antes de produccion)

1. **Cabecera del token**: `Authorization: Bearer` vs `token` .
2. **Ruta login**: `login/Autenticar` vs `login/autenticar` (mayusculas).
3. `baseDatosCliente` : formato/prefijo exacto para Factusol.
4. **Nombre real de la tabla de direcciones** `F_DIR` (inferido) y campo `PAIDIR` .
5. **Campos minimos obligatorios** para crear articulo/cliente/direccion (probados por analogia).
6. **Numeracion de documentos**: si `EscribirRegistro` respeta el contador de serie o hay que calcular `MAX+1` (asumimos manual).
7. Si `OBPRCL` guarda el `CODDIR` numerico o el `NOMDIR` .
8. Tabla de lineas de factura: `F_LFA` vs `F_LFC` .

Prueba minima recomendada: (a) autenticar y fijar la cabecera correcta; (b) `CargaTabla F_ART` (formato de respuesta); (c) `CargaTabla F_DIR` (confirmar nombre de tabla); (d) crear articulo y cliente de prueba (descubrir campos obligatorios); (e) crear un pedido `F_PCL+F_LPC` y verificarlo

en el escritorio de Factusol.